

Der Workflow-Copilot

Wie der Optimierungslauf heute arbeitet — und warum er nie fertig wird

Ursachenanalyse an 31 echten Produktionssitzungen · Stand 30. Juli 2026

Dieses Dokument erklärt in einfachen Worten, was beim autonomen Optimierungslauf passiert, an welchen Stellen er sich selbst blockiert und wie der Ablauf nach den vorgenommenen Korrekturen aussieht. Alle Zahlen stammen aus der Produktionsdatenbank, nicht aus Schätzungen.

Der Befund in einem Satz

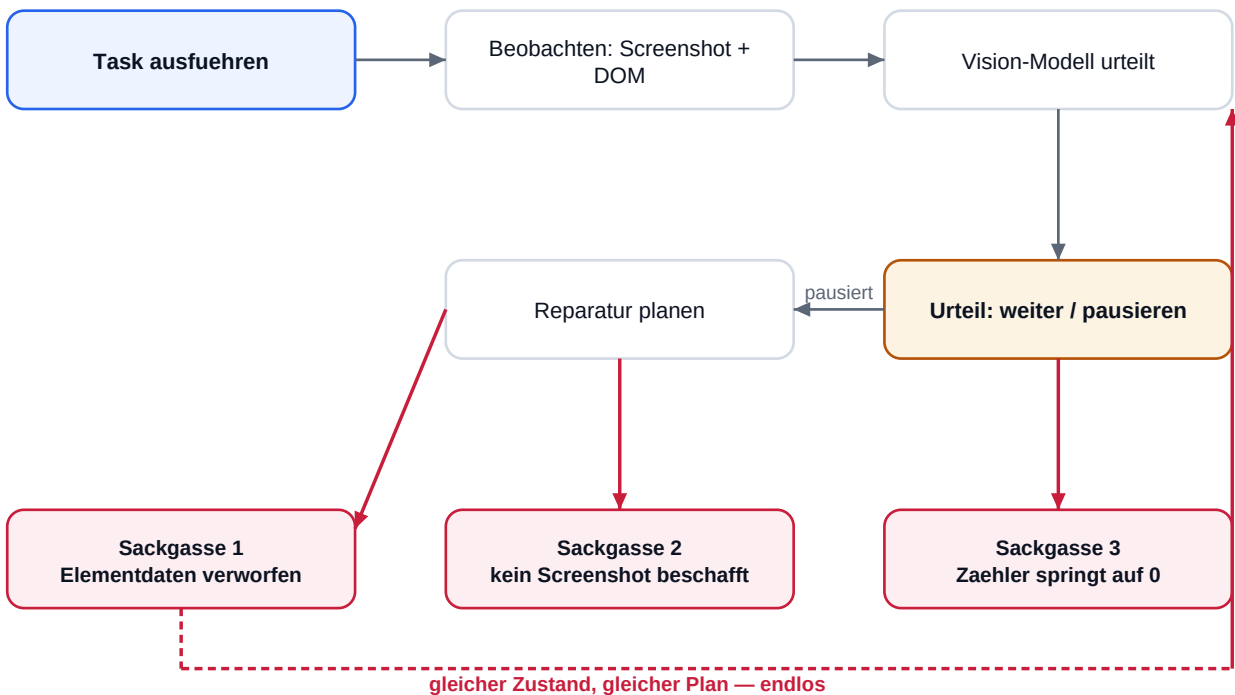
In 31 Sitzungen hat kein einziger Lauf je die Endprüfung bestanden. Keine Sitzung hat sich selbst beendet — jede wurde von Hand abgebrochen, zusammen nach 14,9 Stunden Laufzeit bei nur 2,93 USD Kosten. Es fehlt also nicht an Budget, sondern an Fortschritt.

Die Zahlen

Sitzungen insgesamt	31	alle mit Status „gestoppt“
Erfolgreich abgeschlossen	0	kein verification.passed, kein session.completed
Endprüfung überhaupt erreicht	5 von 31	26 Sitzungen bleiben vorher stecken
Laengste Einzelsitzung	349 Minuten	ohne Selbstabbruch
Reparaturrunden (max.)	12	bei 29 Workflow-Revisionen
Kostenbudget erschöpft	0 mal	Geld war nie die Grenze

1. So laeuft es heute

Der Copilot arbeitet in einer Schleife: Er fuehrt eine Task aus, schaut sich das Ergebnis an, entscheidet und macht weiter. Das Schaubild zeigt den Weg — die roten Pfade sind die drei Sackgassen, in denen die Laeuft tatsaechlich haengenbleiben.



Was in den drei Sackgassen passiert

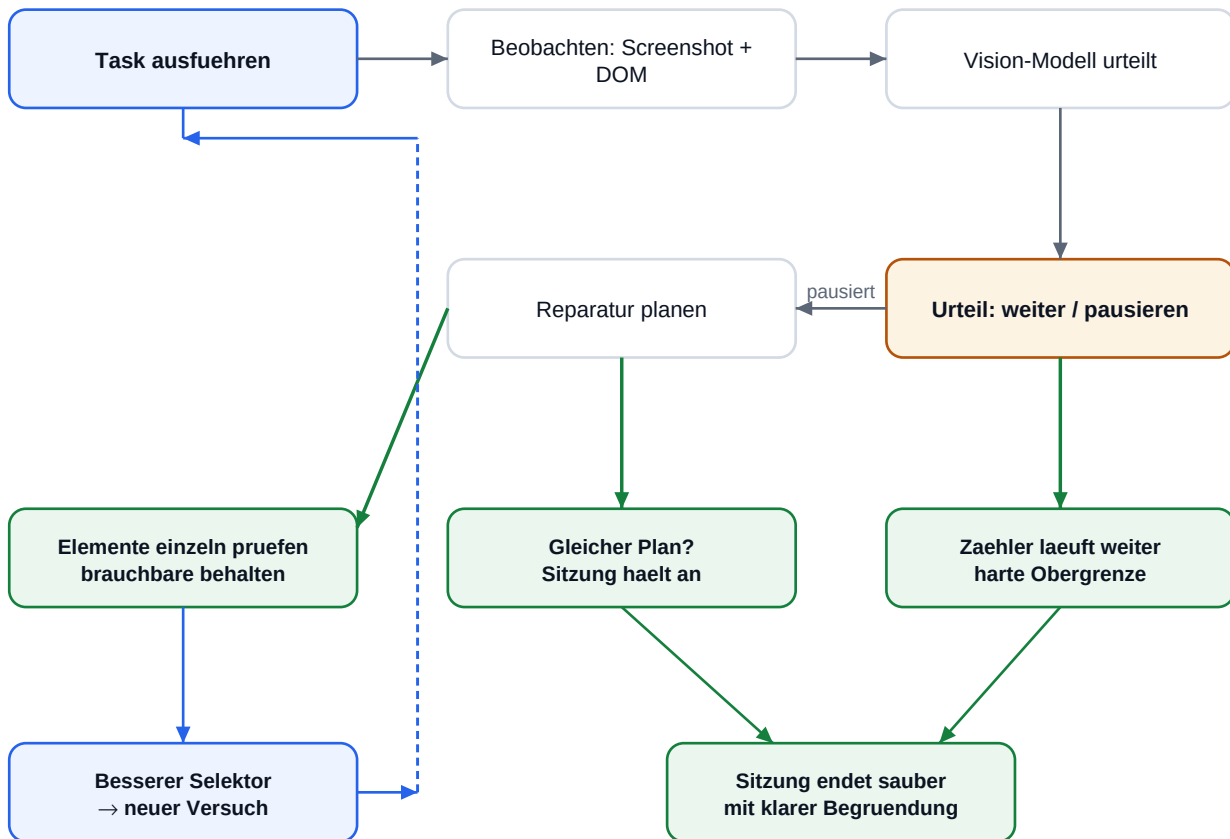
- Sackgasse 1 — Sobald das Vision-Modell „pausieren“ sagt, wurden ALLE erkannten Bedienelemente pauschal verworfen. Genau diese Daten braucht der Copilot aber, um einen besseren Selektor zu finden. Er verliert das Werkzeug in dem Moment, in dem er es braucht.
- Sackgasse 2 — Fehlt ein Screenshot, wird „pausieren“ erzwungen und ein neues Bild verlangt. Nur besorgt niemand eines. Der Lauf wartet auf etwas, das nie kommt.
- Sackgasse 3 — Der Zaehler gegen Endlosschleifen zaehlt nur hoch, solange exakt derselbe Fehler auftritt. Scheitert ein Lauf abwechselnd an zwei Stellen, springt er auf 0 zurueck und die Notbremse greift nie. Zusaetzlich enthielt die Duplikat-Erkennung die Revisionsnummer — die sich bei jeder Reparatur aendert, sodass jeder Plan als „neu“ galt.

Die eigentliche fachliche Ursache

Die letzten sieben Pruefungslaeufe waren technisch einwandfrei und scheiterten nur daran, dass das fachliche Ziel nicht erreicht wurde. Im Klartext aus dem Protokoll: der Klick auf den zu allgemeinen Selektor `button:has-text("Decline")` war technisch erfolgreich, fachlich aber wirkungslos. Genau solche Selektoren kann der Copilot bauartbedingt am schlechtesten reparieren — und die Sackgassen nehmen ihm die Mittel dazu.

2. So laeuft es nach der Optimierung

An drei Stellen im Code wurde eingegriffen. Der Ablauf bleibt derselbe — aber jede Sackgasse hat jetzt einen Ausgang. Gruen markiert, was neu ist.



Die drei Aenderungen im Klartext

- Elementdaten bleiben nutzbar. Statt bei „pausieren“ alles zu verwerfen, wird jedes Bedienelement einzeln gepueft: Ist es im DOM sichtbar, bedienbar, sicher ansprechbar? Die Einzelpruefung war ohnehin strenger als der Pauschalfilter — der Copilot behaelt also seine Reparaturgrundlage, ohne dass die Sicherheit sinkt.
- Wiederholungen werden erkannt. Die Revisionsnummer fliegt aus der Duplikat-Erkennung, und auch ein wiederholtes „pausieren“ zaehlt jetzt als Wiederholung. Derselbe wirkungslose Plan im unveraenderten Zustand fuehrt zu einem klaren Halt statt zu einer weiteren Runde.
- Es gibt eine echte Obergrenze. Ein neuer, nie zuruecksetzender Zaehler begrenzt die Checkpoint-Reparaturen je Sitzung (Standard 15, einstellbar). Damit endet ein aussichtsloser Lauf nach Minuten mit einer nachvollziehbaren Meldung — statt nach Stunden per Hand.

Was das aendert — und was nicht

Die Korrekturen beseitigen die Selbstblockaden und das endlose Kreisen. Sie machen aus einem fachlich falschen Workflow aber keinen richtigen: Wenn ein Selektor nicht zum Ziel fuehrt, meldet der Copilot das jetzt schnell und verstaendlich, statt stundenlang daran zu arbeiten. Der groesste Hebel bleibt, Selektoren vorab gegen die echte Seite zu pruefen.

Empfohlene naechste Schritte

- Selektoren vor dem Optimierungslauf gegen die echte Seite pruefen — sprachunabhaengig und eindeutig statt

Texttreffer wie "Decline".

- Erfolgskriterien in pruefbarer Form angeben. Freitext wie „sassion speichern“ wird stillschweigend verworfen; ohne gueltiges Kriterium gilt die fachliche Pruefung als bestanden, ohne je etwas geprueft zu haben.
- Endpruefung mit eigenem Vision-Prompt versehen (mittlerer Aufwand, noch offen): Sie nutzt heute denselben Text wie die Laufzeitbeobachtung und fragt nach naechsten Schritten statt nach einem Abschlussurteil.